Evolutionary Algorithms: Final report

Your Name (r0123456)

December 17, 2025

Formal requirements

The report is structured for fair and efficient grading of around 100 individual projects in the space of only a few days. Please respect the exact structure of this document. You are allowed to remove this section and 6. Brevity is the soul of wit: a good report will be **around** 8 **pages** long. The hard limit is 10 pages. This **excludes** the other comments in 6 and the generative AI compliance form in 7, which have no page limit.

> Think of this report as a **take-home exam**; it will be used at the exam for structuring the discussion and questions. Make an effort so that it can be visually scanned efficiently, e.g., by using boldface or colors to highlight key points, using lists, clearly defined paragraphs, figures, etc.
>
> You do not need to explain in this report **how** the techniques and concepts that are literally in the slides work. The goal of this report is **not** to illustrate that you can reproduce the slides. You need to convince me that you aptly used these (and other) techniques in this project. If I have doubts about your understanding of certain concepts in the course materials, I will test this hypothesis at the exam.

It is recommended that you use this LaTeX template, but you are allowed to reproduce it with the same structure in a WYSIWYG-editor. The purple text containing our evaluation criteria can be removed. You should replace the blue text with your discussion.

This report should be uploaded to Toledo by December 28, 2025 at 9:00 CET. It must be in the **Portable Document Format** (pdf) and must be named `r0123456_final.pdf`, where r0123456 should be replaced with your student number.

Remove the current section from your final report.

1 Metadata

- **Group members during group phase:** Kaixi Yao, Sarthak Janjuha and Julius Jacobitz
- **Time spent on group phase:** 10 hours
- **Time spent on final code:** 45 hours
- **Time spent on final report:** 10 hours

2 Changes since the group phase **(target:** 0.25 **pages)**

List the main changes that you implemented since the group phase. You do not need to explain the employed techniques in detail; for this, you should refer to the appropriate subsection of section 3 of the report.

1. State here the modification that you made (e.g., replaced top-λ selection with k-tournament selection).
2. Replaced the representation from N-1 cities to N cities always starting at city zero.
3. Replaced random initialization with a greedy initialization on a noisy distance matrix
4. Use OX or EPX as a crossoveroperator with different probability
5. Remove original stopping criteria, instead introduce a nuclear bomb when population has converged.
6. Added local search operators: symmetric 2-opt, symmetric 3-opt, segment-swaps and Or-opt.
7. Added diversity promotion: islands and fitness sharing
8. Adaptively adjust diversity promotion and local search
9. ...

3 Final design of the evolutionary algorithm **(target:** 3.5 **pages)**

Goal: Based on this section, we will evaluate insofar as you are able to design and implement an advanced, effective evolutionary algorithm for solving a model problem.

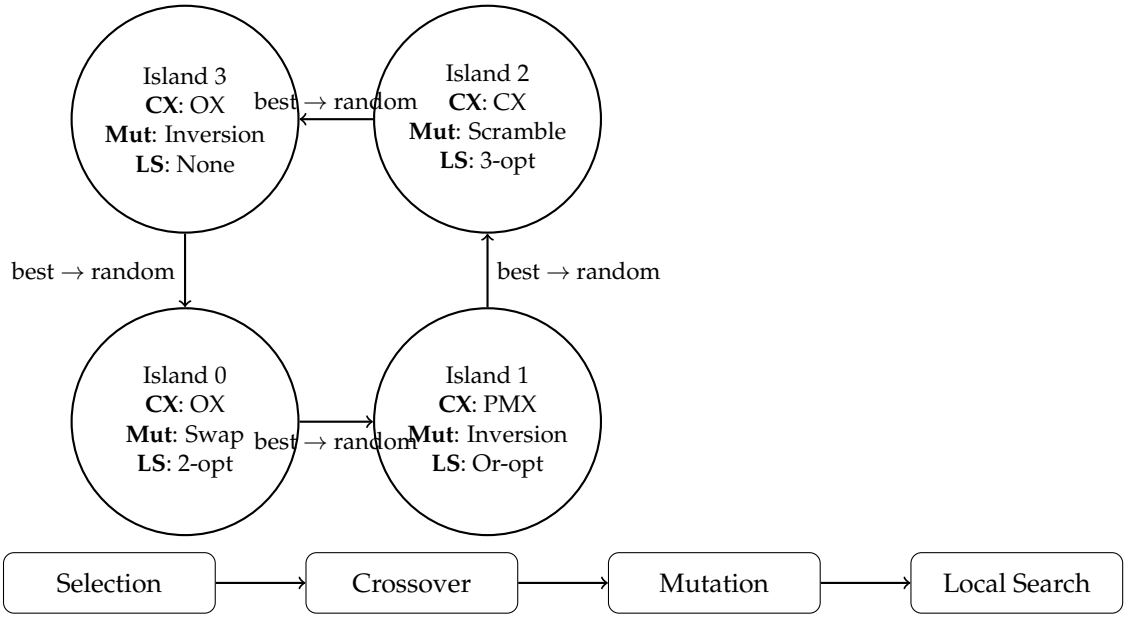
3.1 The three main features

List the three main components of your evolutionary algorithm for this project. That is, what are its most distinctive characteristics, what components am I not allowed to change to a more basic version? Ideally these are some of the more advanced features that you added since the group phase.

1. Just state the feature, you do not need to explain it, instead refer to the appropriate section below.
2. Local search operator: Or-opt
3. ...

3.2 The main loop

Make a picture of the "flow" in your evolutionary algorithm, similar to the example below. Include all the main components (mutation, recombination, selection, elimination, initialization, local search operators, diversity promotion mechanisms). There are no formal requirements on how to do this, as long as it is clear and you can efficiently explain your complete evolutionary algorithm using this picture at the exam. Contrary to the picture below, include the specific techniques, e.g., top-λ elimination, k-tournament selection, where possible.



Island-specific operators

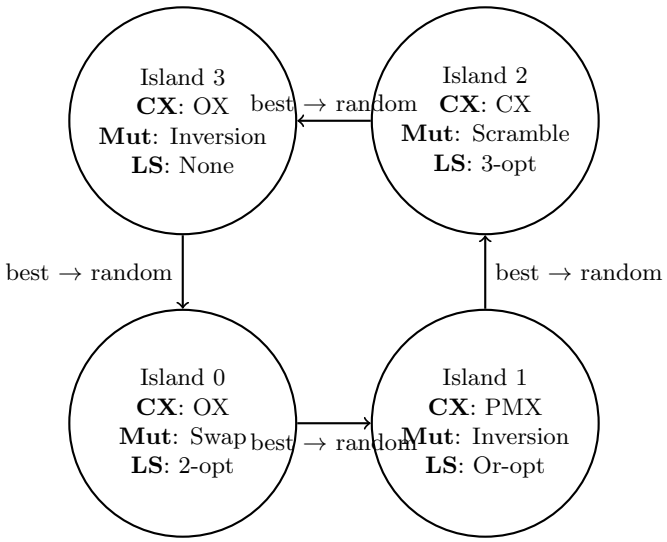


Figure 1: Island-based genetic algorithm with ring migration and heterogeneous operators.

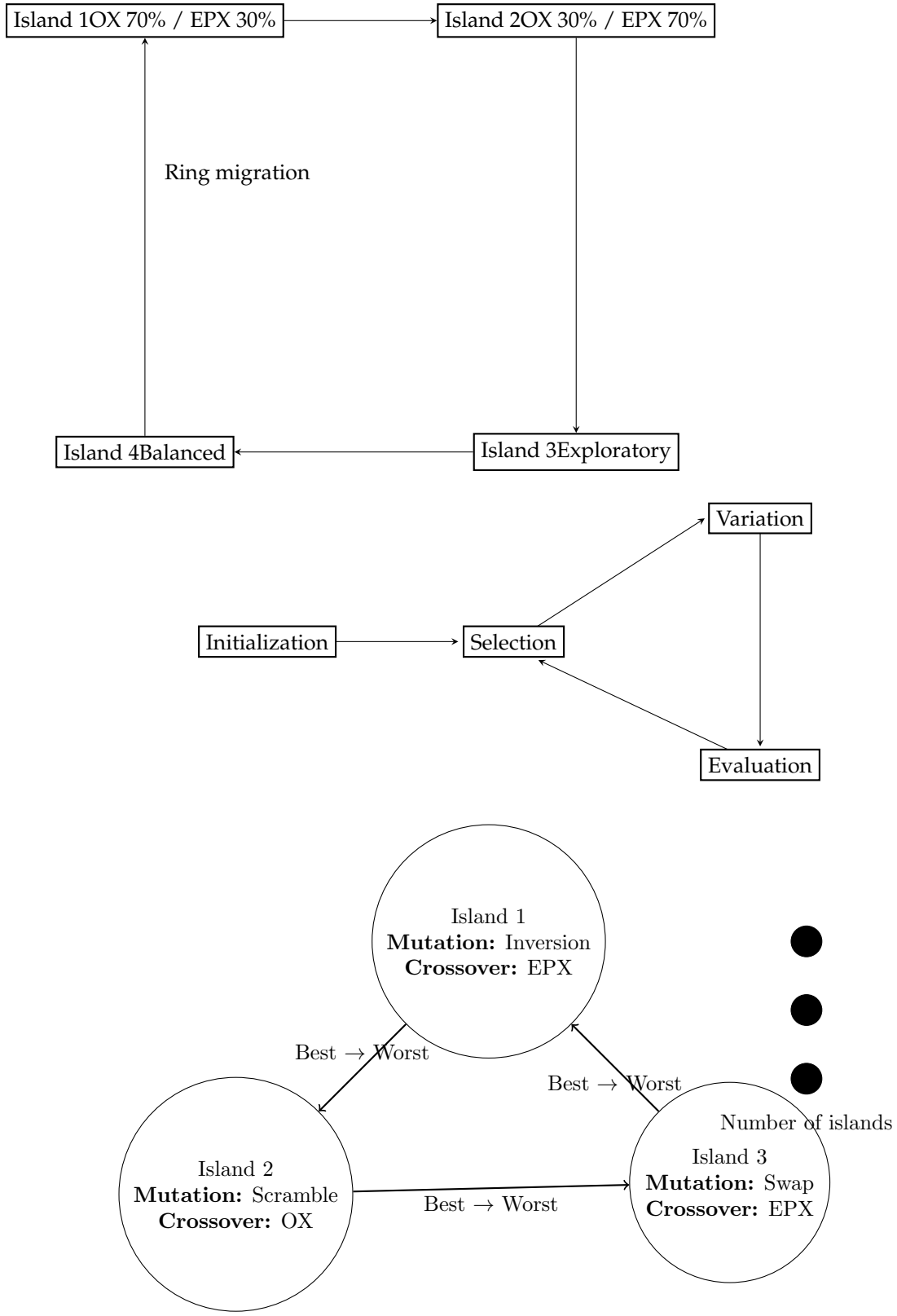


Figure 2: Island-based genetic algorithm with different mutation and crossover types, and migration scheme.

The questions from 3.3 to 3.13 in blue are there to guide which topics to discuss, rather than an exact list of questions that must be answered. Feel free to add more items to discuss.

3.3 Representation

How do you represent the candidate solutions? What is your motivation to choose this one? What other options did you consider? How did you implement this specifically in Python (e.g., a list, set, numpy array, etc)? A candidate solution is represented as a list of cities visited in order. This representation is intuitive and vectorizable if implemented as an array in `numpy`. We make sure that every candidate solution starts with city zero. This allows

us to use *Hamming Distance* as a metric for population diversity. Another way to represent a candidate solution would be to use a binary matrix which indicates which city is visited next. However, for large matrices this will require a lot of memory $O(N^2)$.

In python the Representation is implemented as a numpy array. This choice is preferred due to the compability with `numba` which significantly accelerates computation time.

3.4 Initialization

How do you initialize the population? How did you determine the number of individuals? Did you implement advanced initialization mechanisms (local search operators, heuristic solutions)? If so, describe them. Do you believe your approach maintains sufficient diversity? How do you ensure that your population enrichment scheme does not immediately take over the population? Did you implement other initialization schemes that did not make it to the final version? Why did you discard them? How did you determine the population size?

The population in each island is inialized using a greedy heuristic on a noisy distance matrix. The greedy heuristic allows us to find initial solutions which are better than purely random solutions. Especially for large matrices this initial leap in performance is crucial since purely random initialization will not be able to converge in the allowed time. If the distance matrix contains unconnected cities, the greedy approach allows us to find possible tours faster. We preserve diversity by adding noise on the distance matrix. By varying the amount of noise on this matrix we can control the diversity of the initial population.

BFS and DFS initialization were implemented as well with the idea of finding random initialization which are feasible when the graph is not fully connected. But were left out due to the fact that for large matrices the initial bests were too high to be able to converge leading to limited benefit.

The population size is chosen based on reasoned trial and error. A larger population promotes exploration of the search space, whereas a smaller population emphasizes exploitation of known good solutions.

3.5 Selection operators

Which selection operators did you implement? If they are not from the slides, describe them. Can you motivate why you chose this one? Are there parameters that need to be chosen? Did you use an advanced scheme to vary these parameters throughout the iterations? Did you try other selection operators not included in the final version? Why did you discard them?

The implemented selection operator is k-tournaments since we have direct control on the selection pressure and it works well even when fitness values have a large range (like our tour lengths), as it only compares relative fitness within the tournament group. Small k (e.g., 2-3) means a more diverse selection, weaker individuals have a better chance. Large k (e.g., 5-10) means a more aggressive selection, strongly favoring the best individuals.

3.6 Mutation operators

Which mutation operators did you implement? If they are not from the slides, describe them. How do you choose among several mutation operators? Do you believe it will introduce sufficient randomness? Can that be controlled with parameters? Do you use self-adaptivity? Do you use any other advanced parameter control mechanisms (e.g., variable across iterations)? Did you try other mutation operators not included in the final version? Why did you discard them?

We implemented inversion, scramble and swap mutations. Inversion and scramble mutation are global mutations and the segment length is randomly picked between 0 and the number of cities. Swap mutation is a local mutation. Each island gets a different mixed probabilities of inversion, scramble and swap. In this way different islands become specialist at a certain mutation. This promotes exploration. The diversity between islands can be tuned by a hyperparameter which controls how much mutation mixes are different in each island. The base mutation rate can be tuned as well and can be adaptive. Adaptive mutation

3.7 Recombination operators

Which recombination operators did you implement? If they are not from the slides, describe them. How do you choose among several recombination operators? Why did you choose these ones specifically? Explain how you believe that these operators can produce offspring that combine the best features from their parents. How does your operator behave if there is little overlap between the parents? Can your recombination be controlled with parameters; what behavior do they change? Do you use self-adaptivity? Do you use any other advanced parameter control mechanisms (e.g., variable across iterations)? Did you try other recombination operators not included in the final version? Why did you discard them? Did you consider recombination with arity strictly greater than 2?

We implemented three recombination operators: Ordered Crossover (OX), Edge-Preserving Crossover (EPX), and Edge Recombination Crossover (ERX). OX copies a subsequence from one parent and fills the remaining cities in the order of the second parent, preserving relative city order. EPX and ERX focus on preserving edges from both parents, with ERX building an edge map to guide offspring construction. These operators were chosen because they produce valid tours while retaining promising building blocks, such as good subsequences or edges, increasing the likelihood of high-quality offspring. When parents share few edges, edge-based operators naturally produce more exploratory offspring. Operator selection is probabilistic, and while we currently use fixed probabilities, this could be extended to self-adaptive schemes. Simpler operators like PMX or Cycle Crossover were considered but discarded due to poor edge preservation, and recombination is restricted to two parents for simplicity. To further promote diversity, each island uses a different mix of probabilities for these three operators, allowing islands to specialize in certain crossovers and explore complementary regions of the search space.

3.8 Elimination operators

Which elimination operators did you implement? If they are not from the slides, describe them. Why did you select this one? Are there parameters that need to be chosen? Did you use an advanced scheme to vary these parameters throughout the iterations? Did you try other elimination operators not included in the final version? Why did you discard them?

We implemented (λ, μ) elimination with elitism for our genetic algorithm. Initially, we tested the $(\lambda + \mu)$ scheme, which preserves all elites automatically. However, we observed that this approach led to rapid loss of population diversity, as the best individuals dominated too quickly. To maintain exploration while still retaining the top solutions, we switched to (λ, μ) elimination combined with elitism, which explicitly preserves a small number of the best individuals each generation. This approach allows the majority of the population to be replaced, promoting diversity, while ensuring that the best solutions are never lost. No advanced parameter control (e.g., self-adaptivity) was applied to the elitism fraction.

3.9 Local search operators

What local search operators did you implement? Describe them. Did they cause a significant improvement in the performance of your algorithm? Why (not)? Did you consider other local search operators that did not make the cut? Why did you discard them? Are there parameters that need to be determined in your operator? Do you use an advanced scheme to determine them (e.g., adaptive or self-adaptive)?

We implemented symmetric 2-opt and 3-opt, these method make use of a local delta computation to check which local permutations are better. When the matrix is asymmetric the operator could lead to wrong improvements. Using the delta calculation improves the cost significantly. Or-opt is implemented as well, this method evaluates a subset of 3-opt, it swaps segments without reversing so it is valid for asymmetric matrices. Segment-swaps is considered as well to increase the diversity. We use selfadaptivity.

3.10 Diversity promotion mechanisms

Did you implement a diversity promotion scheme? If yes, which one? If no, why not? Describe the mechanism you implemented. In what sense does the mechanism improve the performance of your evolutionary algorithm? Are there parameters that need to be determined? Did you use an advanced scheme to determine them?

Island model and fitness sharing. Island model with the idea of exploring multiple minima. And fitness sharing to preserve diversity within an island. Fitness sharing is adaptively triggered whenever the hamming distance becomes too low.

3.11 Stopping criterion

Which stopping criterion did you implement? Did you combine several criteria?

We decided that we a stopping criterion won't be the best choice, instead we want to use all five available minutes. Instead we decide a triggering criterion to trigger a nuclear bomb which wipes out the entire population and initializes the population again. This way we run the algorithm again and hope for convergence to a better minimum.

3.12 Parameter selection

For all of the parameters that are not automatically determined by adaptivity or self-adaptivity (as you have described above), describe how you determined them. Did you perform a hyperparameter search? How did you do this? How did you determine these parameters would be valid both for small and large problem instances?

We did do a hyperparameter search with the help of an optimizer tool `optuna`. Optuna builds an internal model of good hyperparameter combinations. By optimizing the parameters for the different tours we arrive at

good hyperparam combinations. Results of optuna are tested and further manually tuned to get idea of what's happening.

3.13 Other considerations

Did you consider other items not listed above, such as elitism, multiobjective optimization strategies (e.g., island model, pareto front approximation), a parallel implementation, or other interesting computational optimizations (e.g. using advanced algorithms or data structures)? You can describe them here or add additional subsections as needed.

4 Numerical experiments (target: 1.5 pages)

Goal: Based on this section and our execution of your code, we will evaluate the performance (time, quality of solutions) of your implementation and your ability to interpret and explain the results on benchmark problems.

4.1 Metadata

What parameters are there to choose in your evolutionary algorithm? Which fixed parameter values did you use for all experiments below? If some parameters are determined based on information from the problem instance (e.g., number of cities), also report their specific values for the problems below.

Report the main characteristics of the computer system on which you ran your evolutionary algorithm. Include the processor or CPU (including the number of cores and clock speed), the amount of main memory, and the version of Python 3.

4.2 tour50.csv

Run your algorithm on this benchmark problem (with the 5 minute time limit from the Reporter). **Include a typical convergence graph, by plotting the mean and best objective values in function of the time** (for example based on the output of the Reporter class).

What is the best tour length you found? What is the corresponding sequence of cities?

Interpret your results. How do you rate the performance of your algorithm (time, memory, speed of convergence, diversity of population, quality of the best solution, etc)? Is your solution close to the optimal one?

Solve this problem 500 times and record the results. Make a histogram of the final mean fitnessess and the final best fitnesses of the 500 runs. Comment on this figure: is there a lot of variability in the results, what are the means and the standard deviations?

4.3 tour250.csv

Run your algorithm on this benchmark problem (with the 5 minute time limit from the Reporter). **Include a typical convergence graph, by plotting the mean and best objective values in function of the time** (for example based on the output of the Reporter class).

What is the best tour length you found in each case?

Interpret your results. How do you rate the performance of your algorithm (time, memory, speed of convergence, diversity of population, quality of the best solution, etc)? Is your solution close to the optimal one?

4.4 tour1000.csv

Answer the same questions as in 4.3.

5 Critical reflection (target: 1.5 page)

Goal: Based on this section, we will evaluate your understanding and insight into the main strengths and weaknesses of evolutionary algorithms, as well as your insights into responsible genAI usage.

5.1 Strengths and weaknesses

What are the three main strengths of evolutionary algorithms in your experience?

1.
2.
3.

What are the three main weak points of evolutionary algorithms in your experience?

1.
2.
3.

Describe the main lessons learned from this project. Do you believe evolutionary algorithms are appropriate for the problem studied in this project? Why (not)? What surprised you and why? What did you learn from this project?

5.2 GenAI peer reviewing critical reflection

What are the strengths and weaknesses of the AI-generated peer review compared to the human-written one? Did Copilot identify issues the humans missed, or vice versa? Why do you think this happened? Was Copilot's review at all useful? Was it specific and concrete or rather vague and general? What is your view towards reviewing reports using generative AI? Would you want to be reviewed by an AI system or by a human? Why? Did writing your own review help you understand the other group's algorithm better than reading the AI's version? How might relying too much on AI affect your ability to critically evaluate scientific work in the future? In what ways can AI be a helpful tool in peer review? In what ways can it be harmful?

5.3 GenAI critical reflection

In what ways did generative AI help you during the project? In what ways did it hinder your learning or understanding? Briefly describe for which activities you believe genAI added value:

1.
2.
3.

For which activities did you not observe any or sufficient added value:

1.
2.
3.

Would your project solution have been substantially different without relying on genAI? In what aspects? Why? How much time did you gain or lose by relying on genAI? If you did not substantially employ genAI: motivate why did you not use it.

6 Other comments (does not count toward page limit)

In case there is something important to discuss that is not covered by the previous sections, you can do it here.

7 Generative AI compliance form (does not count toward page limit)

For **each instance** of a genAI activity that required reporting (generating ideas and code), fill out the following information. You do not need to provide this information for the unconditionally approved uses of genAI.

7.1 Example activity

Briefly describe the activity for which you employed GenAI. For example: "Code skeleton generation for a recombination operator of permutations." Give the subsection an informative title.

Generative AI model: Identify the GenAI model used for this activity. For example, Microsoft Copilot, OpenAI ChatGPT v4, Midjourney, etc.

Motivation: Describe the intention and reason for employing generative AI for this activity.

Methodology: List the exact prompts you used to generate the content. You do not need to include the model's response.

Postprocessing: Describe which postprocessing you applied to the generated content. For example, "the output was used verbatim," "the output contained basic programming errors (undefined variables, undefined functions), which I corrected by hand," "I rewrote/removed/expanded one/a few/some/most/all sentences to [purpose]," "The generated content contained interesting ideas for a recombination operator, suggesting in particular to [do it in such and such way]. However, this idea would have been computationally very expensive, so I simplified [this and that], using [advanced data structure] to obtain an efficient variant of the generated idea."

Reflection: Critically reflect on the output generated and describe what you learned and in which aspects the result was useful and in which aspects it was not useful.

7.2 Another example activity

Generative AI model:

Motivation:

Methodology:

Postprocessing:

Reflection: